Projekt zaliczeniowy – Szkolny Helpdesk

System zgłaszania usterek w szkole

Cel zadania

Waszym zadaniem jest zbudowanie i wdrożenie aplikacji webowej, w której można zgłaszać usterki w szkole (np. niedziałający komputer, zepsuty rzutnik, cieknący kran). Aplikacja ma działać pod publicznym adresem na Waszym VPS, a nie tylko na localhost.

Technologie (obowiązkowo)

- Next.js (App Router, TypeScript) + shadcn/ui
- PocketBase, uruchomiony jako osobna usługa w Coolify
- repozytorium na GitHubie albo w Gitei (patrz niżej)
- Coolify na Waszym VPS

Wymagania – poziom podstawowy

1. Rejestracja i logowanie (PocketBase auth, email i hasło).
2. Dwie role: użytkownik i technik. Rola to pole w kolekcji users, zmienia ją tylko superuser.
3. Użytkownik może:
   a) dodać zgłoszenie: tytuł, opis, sala (lista wyboru), kategoria (sprzęt komputerowy, multimedia, instalacje, inne), priorytet (niski, średni, wysoki),
   b) dołączyć zdjęcie usterki (jeden plik, maks. 5 MB, tylko obrazy),
   c) zobaczyć tylko swoje zgłoszenia i ich status.
4. Technik może:
   a) zobaczyć wszystkie zgłoszenia w tabeli,
   b) filtrować je po statusie, sali i priorytecie,
   c) zmienić status (nowe, w realizacji, zamknięte) i dodać komentarz.
5. Formularze mają walidację (react-hook-form + zod) i czytelne komunikaty o błędach.

Wymagania – poziom rozszerzony

6. Lista zgłoszeń u technika odświeża się sama po dodaniu nowego zgłoszenia (realtime w PocketBase).
7. Historia zmian: osobna kolekcja ticket_events, w której zapisujecie kto, kiedy i z jakiego statusu na jaki zmienił zgłoszenie.
8. Prosty dashboard dla technika: liczba otwartych zgłoszeń, zgłoszenia wg kategorii, sala z największą liczbą zgłoszeń.
9. Tryb jasny i ciemny, aplikacja działa poprawnie na telefonie.

Wymagania techniczne

- Reguły API w PocketBase muszą chronić dane, a nie tylko interfejs. Użytkownik nie może odczytać cudzego zgłoszenia, nawet jeśli wyśle zapytanie ręcznie (curl, konsola

przeglądarki). Będę to sprawdzał.

- Adres PocketBase ustawiacie zmienną środowiskową, nie wpisujecie go na sztywno w kodzie. Pamiętajcie, że zmienne NEXT_PUBLIC_ są wstawiane w czasie buildu.
- PocketBase w Coolify ma trwały wolumen na pb_data. Restart kontenera nie może kasować danych.
- W repozytorium nie ma haseł, tokenów ani pliku .env. Zamiast tego dodajecie .env.example.
- Commity mają sensowne opisy (nie „poprawki”, „asdf”, „final2”).

Repozytorium i wdrożenie

Wybieracie jeden z dwóch wariantów:

Wariant A – GitHub. Repozytorium trzymacie na GitHubie, wdrożenie robicie ręcznie przyciskiem Redeploy w Coolify.

Wariant B – Gitea (dodatkowe 10 pkt). Repozytorium trzymacie w Gitei na VPS i ustawiacie automatyczny deploy: push na gałąź main uruchamia webhook, a Coolify sam buduje i wdraża aplikację. Na prezentacji pokazujecie to na żywo: zmieniacie tekst w aplikacji, robicie push i po chwili zmiana jest widoczna na produkcji.

Co oddajecie

1. Link do działającej aplikacji (HTTPS).
2. Link do repozytorium.
3. Plik README.md, w którym jest: opis projektu ze zrzutami ekranu, instrukcja uruchomienia lokalnie, opis wdrożenia w Coolify (usługi, zmienne środowiskowe, wolumeny) oraz dane kont testowych – jedno konto użytkownika i jedno konto technika.
4. Krótka prezentacja (ok. 5 minut): działanie aplikacji, reguły API i sposób wdrożenia.

Ocenianie

- wymagania podstawowe (1–5): 35 pkt
- wymagania rozszerzone (6–9): 20 pkt
- bezpieczeństwo (reguły API, brak sekretów w repozytorium): 15 pkt
- wdrożenie (działa na produkcji, dane się nie kasują, zmienne środowiskowe): 15 pkt
- jakość kodu, commity, README: 10 pkt
- prezentacja: 5 pkt
- wariant B (Gitea z automatycznym deployem): dodatkowo 10 pkt

Progi: dopuszczający od 40 pkt, dostateczny od 55 pkt, dobry od 70 pkt, bardzo dobry od 85 pkt, celujący od 95 pkt.

Termin oddania: .....

Forma pracy: .....